



CS1004 - Object Oriented Programming

LAB # 4

RECURSION

Lab Objectives

- Understand the concept of recursion and recursive function calls
- Master base case and recursive case identification
- Learn to trace recursive function execution and call stack
- Implement mathematical recursive functions (factorial, fibonacci, power)
- Apply recursion to array and string manipulation problems
- Understand the difference between iteration and recursion
- Solve problems using divide-and-conquer approach
- Analyze time and space complexity of recursive algorithms
- Practice tail recursion optimization techniques
- Debug and optimize recursive solutions

CONCEPT REVIEW

1. What is Recursion?

Recursion is a programming technique where a function calls itself to solve a problem. A recursive solution breaks a complex problem into smaller, similar sub-problems until it reaches a simple base case that can be solved directly.

Key Components of Recursion:

- Base Case: The stopping condition that prevents infinite recursion
- Recursive Case: The part where the function calls itself with modified parameters
- Progress Toward Base Case: Each recursive call must move closer to the base case

Example:

```
void countdown(int n) {  
    if (n == 0) {        // Base case
```

```
    cout << "Done!";  
    return;  
}  
cout << n << " ";  
countdown(n - 1);    // Recursive case  
}
```

2. How Recursion Works - The Call Stack

When a function calls itself, each call is placed on the call stack. The stack stores:

- Function parameters and local variables
- Return address (where to continue after the function returns)
- The state of the function at that point

Execution Flow:

```
factorial(3)  
→ 3 * factorial(2)  
→ 2 * factorial(1)  
→ 1 * factorial(0)  
→ returns 1  
→ returns 1 * 1 = 1  
→ returns 2 * 1 = 2  
→ returns 3 * 2 = 6
```

3. Base Case - The Foundation of Recursion

The base case is the terminating condition that stops recursion. Without a proper base case, the function will call itself infinitely, leading to a stack overflow error.

Rules for Base Cases:

- Must be reachable: Every execution path should eventually hit a base case
- Must be simple: Should not involve recursive calls
- Must return a value: Should provide the answer for the simplest problem
- May be multiple: Complex problems may need several base cases

Example:

```
int factorial(int n) {  
    if (n <= 1)    // Base  
        case return 1;  
    return n * factorial(n - 1);  
}
```

4. Recursive Case - Breaking Down the Problem

The recursive case defines how the problem is divided into smaller sub-problems. It must satisfy two requirements:

- Make progress: Each recursive call must move toward the base case
- Solve sub-problem: Combine the result of the recursive call to solve the current problem

Example - Computing Power:

```
int power(int base, int exp) {
    if (exp == 0)           // Base case
        return 1;
    return base * power(base, exp-1); // Recursive case
}

// power(2, 3) = 2 * power(2, 2)
//           = 2 * 2 * power(2, 1)
//           = 2 * 2 * 2 * power(2, 0)
//           = 2 * 2 * 2 * 1 = 8
```

5. Recursion vs Iteration

Both recursion and iteration can solve the same problems, but they have different characteristics:

Recursion	Iteration
Uses function calls	Uses loops (for, while)
Uses call stack memory	Uses loop variables
More memory overhead	Less memory usage
Can be more elegant/readable	Usually more efficient
Risk of stack overflow	No stack overflow risk
Natural for tree/divide-conquer	Natural for sequential operations

When to use Recursion:

- Problem has a naturally recursive structure (trees, graphs)
- Problem can be divided into similar sub-problems
- Code clarity is more important than performance
- Divide-and-conquer approach is suitable

6. Types of Recursion

a) Direct Recursion

A function calls itself directly.

```
int factorial(int n) {  
    if (n <= 1) return 1;  
    return n * factorial(n - 1); // Calls itself  
}
```

b) Indirect Recursion

Function A calls function B, which calls function A.

```
void functionA(int n) {  
    if (n > 0) {  
        cout << n << " ";  
        functionB(n - 1);  
    }  
}  
  
void functionB(int n) {  
    if (n > 0) {  
        cout << n << " ";  
        functionA(n - 1);  
    }  
}
```

c) Tail Recursion

The recursive call is the last operation in the function (can be optimized by compiler).

```
int factorial(int n, int result = 1) {  
    if (n <= 1) return result;  
    return factorial(n - 1, n * result); // Last operation  
}
```

d) Non-Tail Recursion

Operations are performed after the recursive call returns.

```
int factorial(int n) {  
    if (n <= 1) return 1;  
    return n * factorial(n - 1); // Multiplication after return  
}
```

7. Common Recursive Patterns

a) Linear Recursion - Single recursive call

```
int sum(int n) {  
    if (n == 0) return 0;
```

```
    return n + sum(n - 1);  
}
```

b) Binary Recursion - Two recursive calls

```
int fibonacci(int n) {  
    if (n <= 1) return n;  
    return fibonacci(n - 1) + fibonacci(n - 2);  
}
```

c) Multiple Recursion - More than two recursive calls

```
void printCombinations(string prefix, string remaining) {  
    if (remaining.empty()) {  
        cout << prefix << endl;  
        return;  
    }  
    for (int i = 0; i < remaining.size(); i++) {  
        printCombinations(prefix + remaining[i],  
                           remaining.substr(0,i) + remaining.substr(i+1));  
    }  
}
```

8. Recursion with Arrays

Recursion is powerful for array operations. Common pattern: process one element and recurse on the rest.

Example - Sum of Array:

```
int arraySum(int arr[], int n) {  
    if (n == 0) // Base case: empty  
        array return 0;  
    return arr[n-1] + arraySum(arr, n-1); // Last element + sum of rest  
}
```

Example - Find Maximum:

```
int findMax(int arr[], int n) {  
    if (n == 1) // Base case: one  
        element return arr[0];  
    int maxRest = findMax(arr, n-1);  
    return (arr[n-1] > maxRest) ? arr[n-1] : maxRest;  
}
```

9. Recursion with Strings

String problems often have elegant recursive solutions.

Example - Reverse String:

```
void reverseString(string str, int start, int end) {  
    if (start >= end)        // Base case  
        return;  
    swap(str[start], str[end]);  
    reverseString(str, start+1, end-1); // Recurse on middle  
}
```

Example - Check Palindrome:

```
bool isPalindrome(string str, int start, int end) {  
    if (start >= end)        // Base case: checked all  
        return true;  
    if (str[start] != str[end]) // Mismatch found  
        return false;  
    return isPalindrome(str, start+1, end-1);  
}
```

10. Tracing Recursive Calls

Tracing helps understand recursion by showing the sequence of calls and returns.

Example: Tracing factorial(4)

```
Call Stack Trace:  
factorial(4)  
    → 4 * factorial(3)  
        → 3 * factorial(2)  
            → 2 * factorial(1)  
                → return 1    [Base case reached]  
            → return 2 * 1 = 2  
        → return 3 * 2 = 6  
    → return 4 * 6 = 24
```

Final Result: 24

Tips for Tracing:

- Write down each function call with its parameters
- Indent each level of recursion
- Mark when base case is reached
- Track return values back up the call stack

- Count the number of recursive calls

11. Common Recursion Errors

a) Missing Base Case

```
// WRONG - Infinite recursion  
int factorial(int n) {  
    return n * factorial(n - 1); // No base case!  
}
```

b) Base Case Never Reached

```
// WRONG - n never reaches 0  
int factorial(int n) {  
    if (n == 0) return 1;  
    return n * factorial(n + 1); // Going wrong direction!  
}
```

c) Not Making Progress

```
// WRONG - Same parameter every time  
int factorial(int n) {  
    if (n == 0) return 1;  
    return n * factorial(n); // n doesn't change!  
}
```

12. Time and Space Complexity

Recursive functions have both time complexity (number of operations) and space complexity (call stack depth).

Examples:

Function	Time Complexity	Space Complexity
factorial(n)	$O(n)$	$O(n)$
fibonacci(n) naive	$O(2^n)$	$O(n)$
binarySearch(n)	$O(\log n)$	$O(\log n)$
sum of array(n)	$O(n)$	$O(n)$



LAB TASKS

Task 1: Print N to 1 and 1 to N

Write recursive functions to print numbers from N to 1 and 1 to N.

Requirements:

- Function 1: void printDescending(int n) - prints N to 1
- Function 2: void printAscending(int n) - prints 1 to N
- Both must use recursion
- No loops allowed

Sample Input:

n = 5

Sample Output:

Descending: 5 4 3 2 1

Ascending: 1 2 3 4 5

Task 2: Sum of Even and Odd Numbers

Write recursive functions to find sum of even and odd numbers from 1 to N.

Requirements:

- Function 1: int sumEven(int n)
- Function 2: int sumOdd(int n)
- Display both sums
- Verify results manually

Sample Input:

n = 10

Sample Output:

Sum of even numbers (2+4+6+8+10): 30

Sum of odd numbers (1+3+5+7+9): 25

Task 3: Search Element in Array

Implement linear search using recursion.

Requirements:

- Function signature: int linearSearch(int arr[], int n, int target)



- Return index if found, -1 if not found
- Display search progress
- Base cases: $n=0$ (not found) or element found

Sample Input:

Array: [10, 23, 45, 70, 11, 15]
Target: 70

Sample Output:

Checking index 0: 10
Checking index 1: 23
Checking index 2: 45
Checking index 3: 70 - Found!
Element found at index: 3

Task 4: Count Consonants in String

Count consonants (non-vowel letters) in a string recursively.
Requirements:

- Function signature: `int countConsonants(string str, int index)`
- Consider both uppercase and lowercase
- Ignore non-alphabetic characters
- Display count

Sample Input:

`str = "Hello World 123"`

Sample Output:

Consonants: 7 (H, l, l, W, r, l, d)

Task 5: Reverse an integer

Write a recursive function to reverse the digits of an integer. Requirements:

- Function signature: `int reverseInt(int n, int rev)`
- Handle both positive and negative numbers
- Use helper parameter for building reversed number
- Display step-by-step reversal

Sample Input: `n = 12345`

Sample Output: Reversed: 54321



Task 6: Count Paths in Grid

Count number of paths from top-left to bottom-right in a grid (can only move right or down).

Requirements:

- Function signature: `int countPaths(int m, int n)`
- `m` = rows, `n` = columns
- Can only move right or down
- Count all possible paths

Display path count

Sample Input:

`m = 3, n = 3 (3x3 grid)`

Sample Output:

Number of paths: 6

